

Object-Oriented Concepts with UML Inheritance

CCP6224

Object-oriented Analysis and Design

Lecture outcomes

- At the end of this lecture, you should
 - Learn about class hierarchy and how inheritance is provided in code
 - Learn **final**, **@override**, **extends**, **implements** and **super** keywords and how they are used
 - Understand how abstract and interface classes are implemented and know the differences/similarities between them
 - Know how substitution in objects work with upcasting and downcasting

Creating new objects

- OOP recommends reuse of existing objects whenever possible – try not to reinvent the wheel!
- There are two ways of creating new objects by reusing existing objects – composition and inheritance
- With composition, you create a new class, which is composed of existing classes – i.e. other classes become ‘data types’ in the new class
- With inheritance, you create a new class based on an existing class, with some modifications or extensions.
- There are no hard and fast rules, but the recommendation is to use composition whenever possible before considering inheritance.

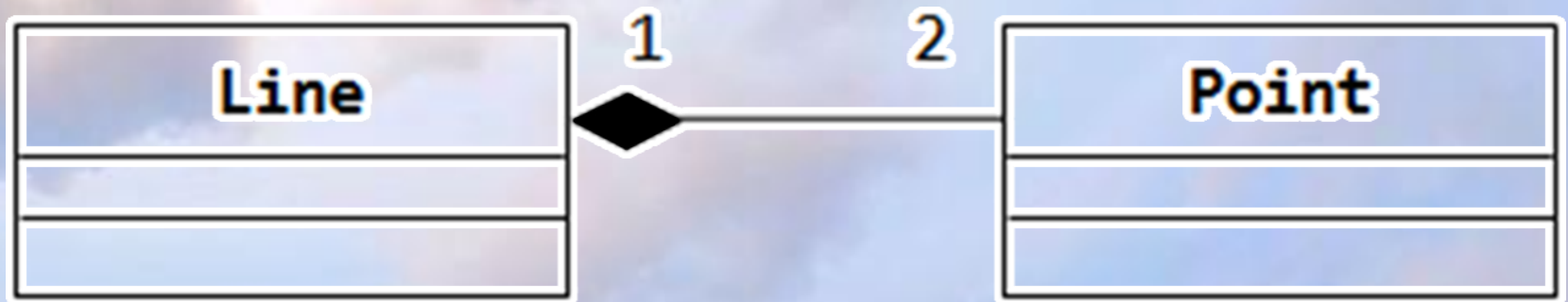
Object composition

- As an example of reusing a class via composition, suppose that we have an existing class called **Point**, defined as follows

Point
-x:int -y:int
+Point() +Point(x:int, y:int) +getX():int +setX(x:int):void +getY():int +setY(y:int):void +toString():String

```
public class Point {  
    private int x, y;  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public Point() {  
        x = 0;  
        y = 0;  
    }  
    public int getX() { return x; }  
    public void setX(int x) { this.x = x; }  
    public int getY() { return y; }  
    public void setY(int y) { this.y = y; }  
    public String toString() {  
        return "(" + x + "," + y + " )";  
    }  
}
```

- Composition exhibits a “has-a” relationship.
- In UML notation, composition is represented as a diamond-head line pointing to its constituents.
- Suppose that we want to create a new class called **Line**, we can reuse the **Point** class via composition → defining "a line is composed of two points" or "a line has two points".

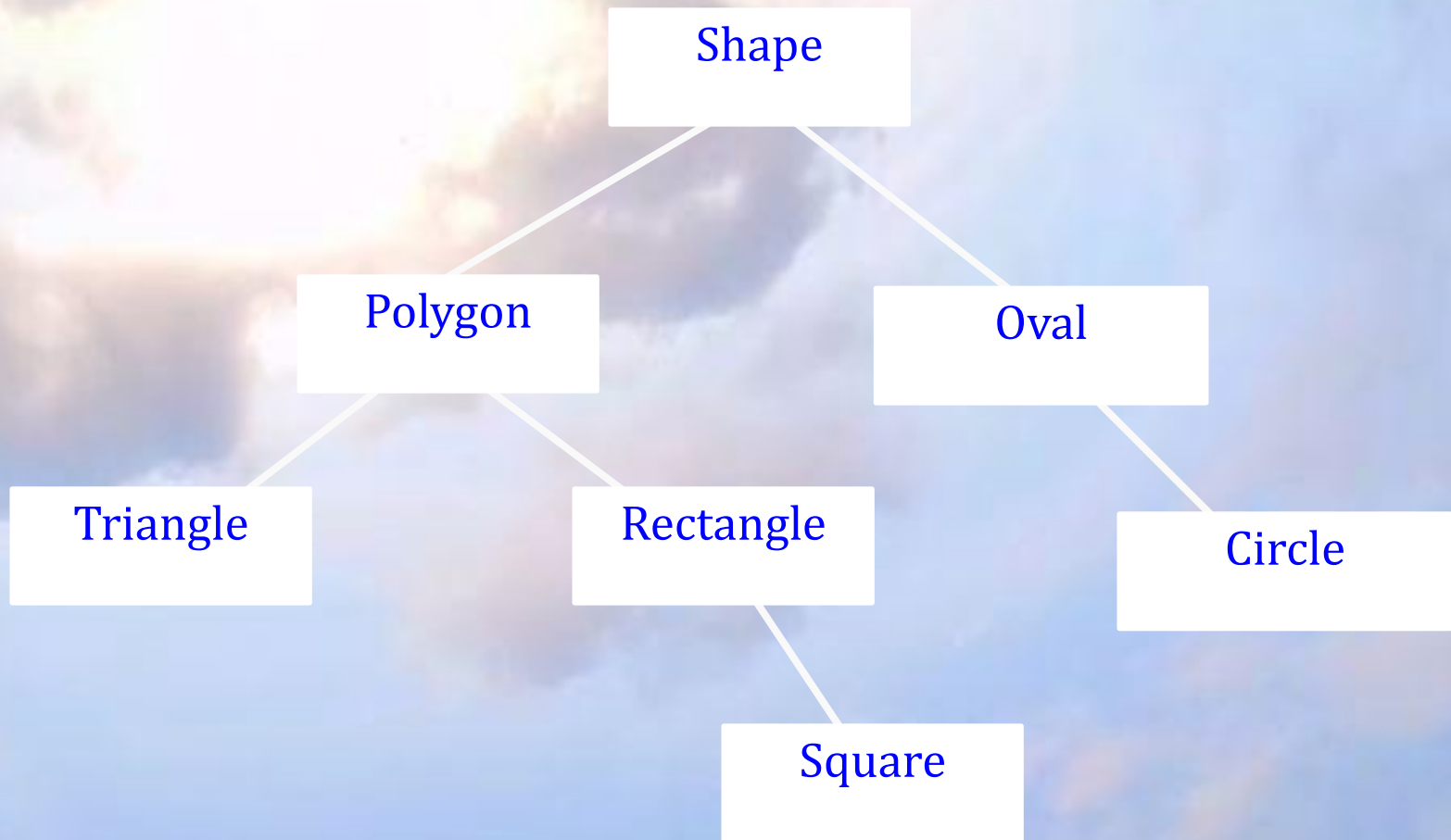


- The new **Line** class is composed (hence *composition*) of two **Point** objects (begin and end)

```
public class Line {  
    Point begin, end;  
    public Line(int x1, int y1, int x2, int y2) {  
        begin = new Point(x1, y1);  
        end    = new Point(x2, y2);  
    }  
    public Line(Point begin, Point end) {  
        this.begin = begin;  
        this.end    = end;  
    }  
    public String toString() {  
        return "Line from " + begin + " to " + end;  
    }  
}
```

Object inheritance

- In OOP, it is recommended to organize classes in hierarchy to avoid duplication and reduce redundancy – this naturally encourages inheritance
- Inheritance facilitates code reuse, and rapid prototyping.
- The classes in the lower hierarchy inherit all the variables (static attributes) and methods (dynamic behaviors) from the higher hierarchies.
- A class in the lower hierarchy is called a subclass (or derived, child, extended class).
- Inheritance provides ability to create a new subclass which is a *variant* of an original superclass – it cannot use composition because it is not exhibiting *has-a* properties but more of a *is-a* relationship



- A class in the upper hierarchy is called a superclass (or alternatively as base or parent class).
- By pulling out all the common variables and methods into the superclasses and leaving the specialized variables and methods in the subclasses, redundancy can be greatly reduced or eliminated as these common variables and methods do not need to be repeated in all the subclasses
- Instances of a subclass can access both data and methods associated with the superclass.
- Inheritance is transitive, so that a sub- class can inherit features from superclasses many levels away.

Inheritance in Java

- To inherit from a superclass, a subclass must use the **extends** keyword and refer to the superclass name

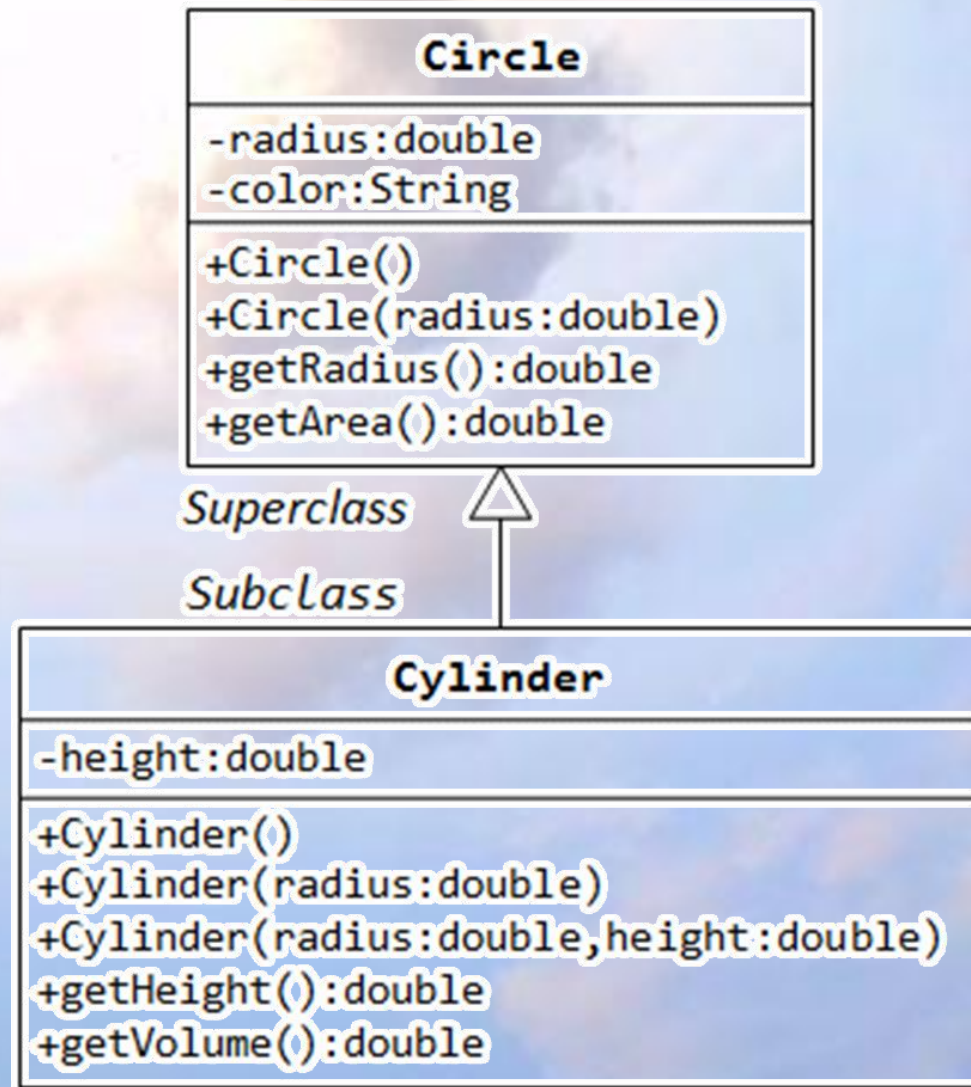
```
class SubClass extends SuperClass{...}
```

- Examples

```
class Goalkeeper extends SoccerPlayer {...}
```



- `class Cylinder extends Circle {...}`



Constructor inheritance

- A subclass does not inherit the constructor from its superclass (because it does not have the same class name!!)
- BUT a subclass *will* invoke the superclass constructor

```
class Animal{  
    Animal() {  
        System.out.println("Hello! I'm an animal");  
    }  
}
```

```
class Cat extends Animal{  
    Cat() {  
        System.out.println("Meow! I'm a Cat");  
    }  
}
```

```
class prog {  
    public static void main(String[] a){  
        Cat whiskas = new Cat();  
    }  
}
```

C:\Windows\system32\cmd.exe

```
D:\>java prog  
Hello! I'm an animal.  
Meow! I'm a Cat  
D:\>_
```

Invoking superclass constructor

- A subclass will auto-invoke the default constructor of its superclass.
- If the superclass has overloaded constructors, a specific one could be invoked by passing relevant arguments (to trigger matching constructor) to the **super** method.

```
class Animal{
    Animal() {
        System.out.println("Hello! I'm an animal");
    }
    Animal(String name){
        System.out.println("Hello! I'm " + name);
    }
}

class Cat extends Animal{
    Cat(String name){
        super(name);
        System.out.println("Meow! I'm a Cat");
    }
}
```

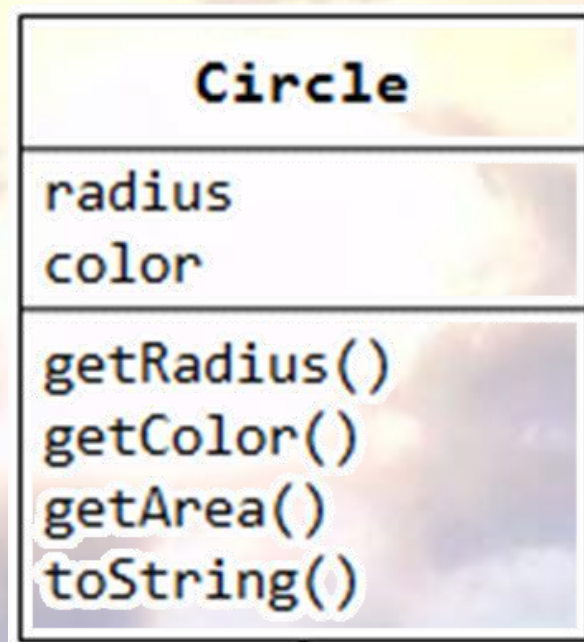
```
public class Cylinder extends Circle {
    private double height;          // member variable

    public Cylinder() {
        super(); // invoke superclass' constructor Circle()
        height = 1.0;
    }
    public Cylinder(double radius, double height) {
        super(radius); // invoke superclass' Circle(radius)
        this.height = height;
    }
    public double getHeight() {
        return height;
    }
    public void setHeight(double height) {
        this.height = height;
    }
    public double getVolume() {
        return getArea()*height; // Use Circle's getArea()
    }
}
```

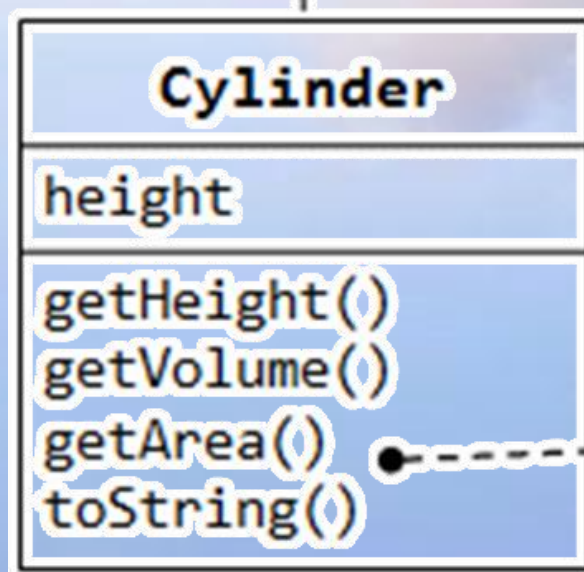
Overriding methods

- A subclass inherits all the member variables and methods from its superclasses (the immediate parent and all its ancestors).
- It can use the inherited methods and variables as they are. It may also override an inherited method by providing its own version, or hide an inherited variable by defining a variable of the same name.
- Overriding only occurs between classes (e.g. subclass inherits from another superclass)
- For example, the inherited method **getArea()** in a Cylinder object computes the base area of the cylinder.
- Suppose that you decide to override the **getArea()** to compute the surface area of the cylinder in the class Cylinder.

```
public double getArea() {  
    return 2*Math.PI*getRadius()* height +  
           2*super.getArea();  
}
```



Superclass



Subclass

Override getArea()
& toString()

@Override

- The **@Override** keyword is known as annotation which asks the compiler to check whether there is such a method in the superclass to be overridden.
- This helps greatly if you misspell the name of the method to be overridden.
- For example, if **@Override** is NOT used and **toString()** is misspelled as **TOSTring()**, it will be treated as a *new* method in the subclass, instead of overriding the superclass.
- If **@Override** is used, the compiler would flag the code stating that a matching method could not be found to be overridden.
- Syntax:

```
@Override  
public int aFunction() {  
    . . .  
}
```

Super

- Inheritance allows subclass to inherit public and protected attributes and methods from the superclass
- When a method is called from a subclass, the methods within the subclass are always called first. If no method exists, then the superclass is checked to see if the method exists – repeat until method is found, else terminate with error.
- To purposely invoke superclass' method, use the keyword **super**
- If a subclass overrides a method inherited from its superclass, say **getArea()**, you can use **super.getArea()** to invoke the superclass' version within the subclass definition
- Similarly, if your subclass hides one of the superclass' variable, you can use **super.variableName** to refer to the hidden variable within the subclass definition
- Without any method identifier, **super()** and **super(argumentList)** can be used invoke the superclass' constructor – as shown earlier

Polymorphism

- Poly = many
- Morph = shape
- Same method, same parameters, but objects of different classes.
- Don't have to know what actual class it is – only that it understands the method.



Overloading

- Overloading is when the same name is used for methods that have the same name but a different signature.

```
void print(String s) {...}
```

```
void print(int i) {...}
```

```
void print(double x) {...}
```

```
void print(int i, int j) {...}
```

- A signature refers to the method's parameter list – to be unique they can either differ in number, type and/or sequence.
- The method's return type does not distinguish a method to be unique

- Overloading also occurs as a result of inheritance (although it may be less obvious because the overloaded methods may not be from the same class)

```
class Window{
    ...
    void show(int x) {
        ...
    }
}

class MenuWindow
    extends Window{
    ...
    void show() {
        ...
    }
}

Window w = new Window();
...
w.show(3);

MenuWindow m = new
    MenuWindow();
...
w = m;
w.show();
```

Properties of inheritance

- A class in Java can only inherit (**extends**) from one (1) superclass only – there is no support for multiple inheritance.
- Subclass and superclasses may be used to represent each other in a practise called substitution.
- Substitutability can occur when a subclass instance substitutes a superclass (upcasting) or vice-versa (downcasting)
- Basic substitution rule: A subclass object can be used in place of a superclass object.

```
Rectangle r = new Rectangle();
```

```
Square s = new Square();
```

```
r = s;
```

Upcasting

- Substituting a subclass instance for its superclass is called "upcasting".
- Upcasting is always safe because a subclass instance possesses all the properties of its superclass and can do whatever its superclass can do.
- Via substitutability, we can create an instance of Cylinder (subclass), and assign it to a Circle (its superclass) reference, as follows:

```
Circle aCircle = new Cylinder(5.0);
```

- You can invoke all the methods defined in the Circle class for the reference aCircle, (which is actually holding a Cylinder object), e.g. **aCircle.getRadius()** and **aCircle.getColor()**
- You cannot invoke methods defined in the Cylinder class for the reference aCircle, e.g. **aCircle.getHeight()** and **aCircle.getVolume()**
- This is because aCircle is a reference to the Circle class.

Downcasting

- Opposite of upcasting - substituting a superclass in a subclass instance
- USUALLY NOT RECOMMENDED
- Downcasting is not always safe, and throws a runtime `ClassCastException` if the instance to be downcasted does not belong to the correct subclass. A subclass object can be substituted for its superclass, but the reverse is not always true.
- Requires casting operator to inform compiler the class type to downcast to (else compile-time error will occur)

```
Cylinder aCylinder = (Cylinder) aCircle;
```

- Downcasting exists to allow extended classes to be compared (e.g. to check inheritance line)

Abstract classes

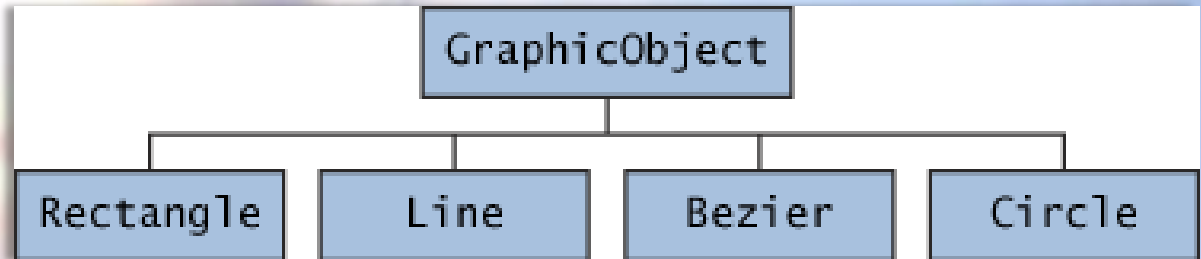
- An *abstract class* is a class that is declared **abstract** — it may or may not include abstract methods.
- If any class has a method declared as abstract, the class **MUST** be abstract
- Abstract methods are methods which are declared but with no implementation defined (no method body).
- Abstract classes cannot be instantiated, but they can be subclassed - The subclass must implement/define all abstract methods inherited from the superclass, else declare itself abstract.

```
public abstract class GraphicObj{  
    // declare fields  
    // declare nonabstract methods  
    abstract void draw();  
}
```

```
public abstract class GraphicObj2 {  
    // declare fields  
    // declare nonabstract methods  
}
```

Abstract class example

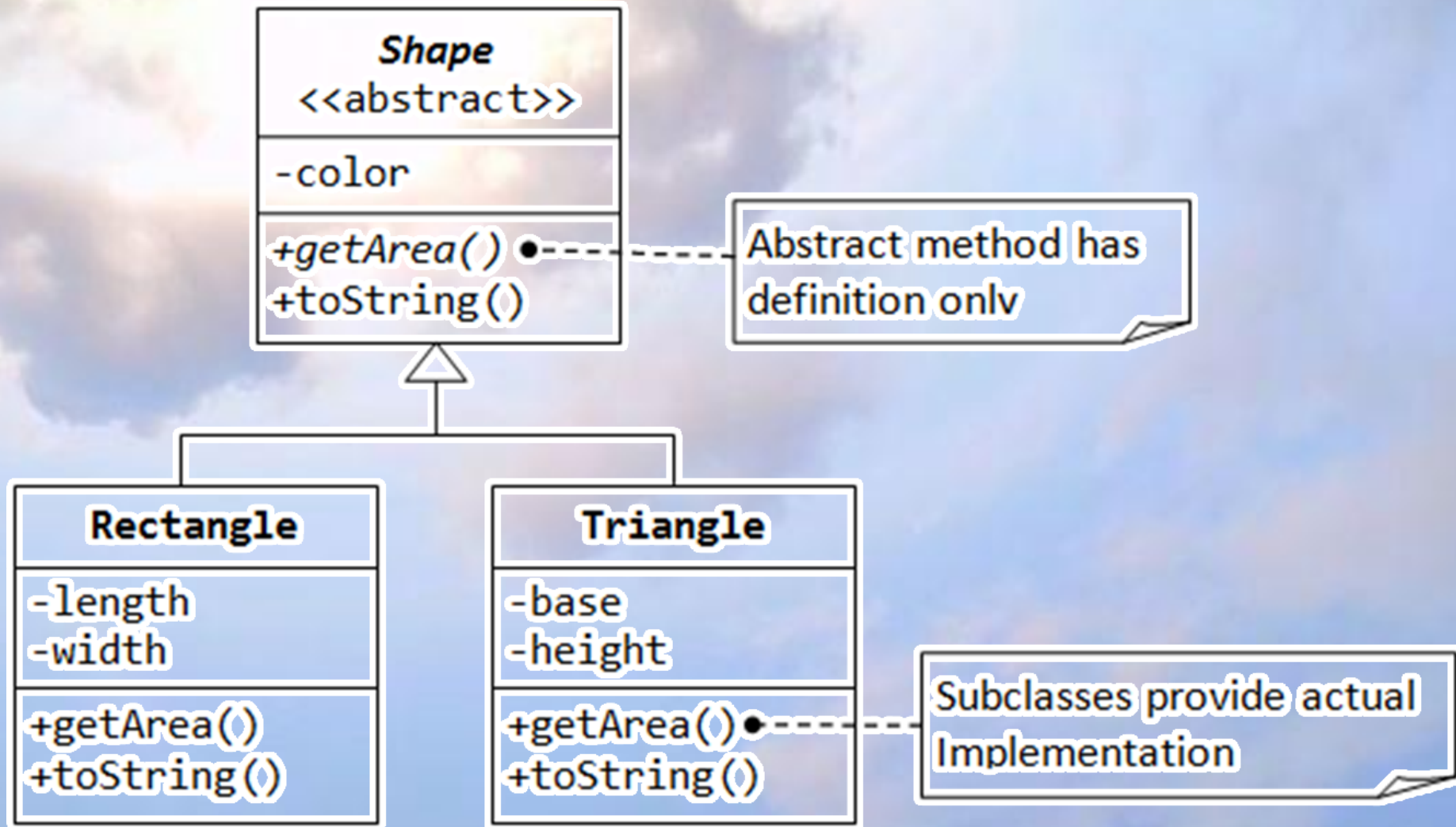
```
class Circle extends GraphicObject {  
    void draw() {  
        ...  
    }  
    void resize() {  
        ...  
    }  
}
```



```
class Rectangle extends GraphicObject {  
    void draw() {  
        ...  
    }  
    void resize() {  
        ...  
    }  
}
```

```
abstract class GraphicObject {  
    int x, y;  
    ...  
    void moveTo(int newX, int newY) {  
        ...  
    }  
    abstract void draw();  
    abstract void resize();  
}
```

Abstract class example 2



```
public abstract class Shape {  
    // Instance variable  
    private String color;  
  
    // Constructor  
    public Shape (String color) {  
        this.color = color;  
    }  
  
    public String toString() {  
        return "Shape of color=\"" + color + "\"";  
    }  
  
    // All subclasses must implement a method called getArea()  
    abstract public double getArea();  
}
```

Interface

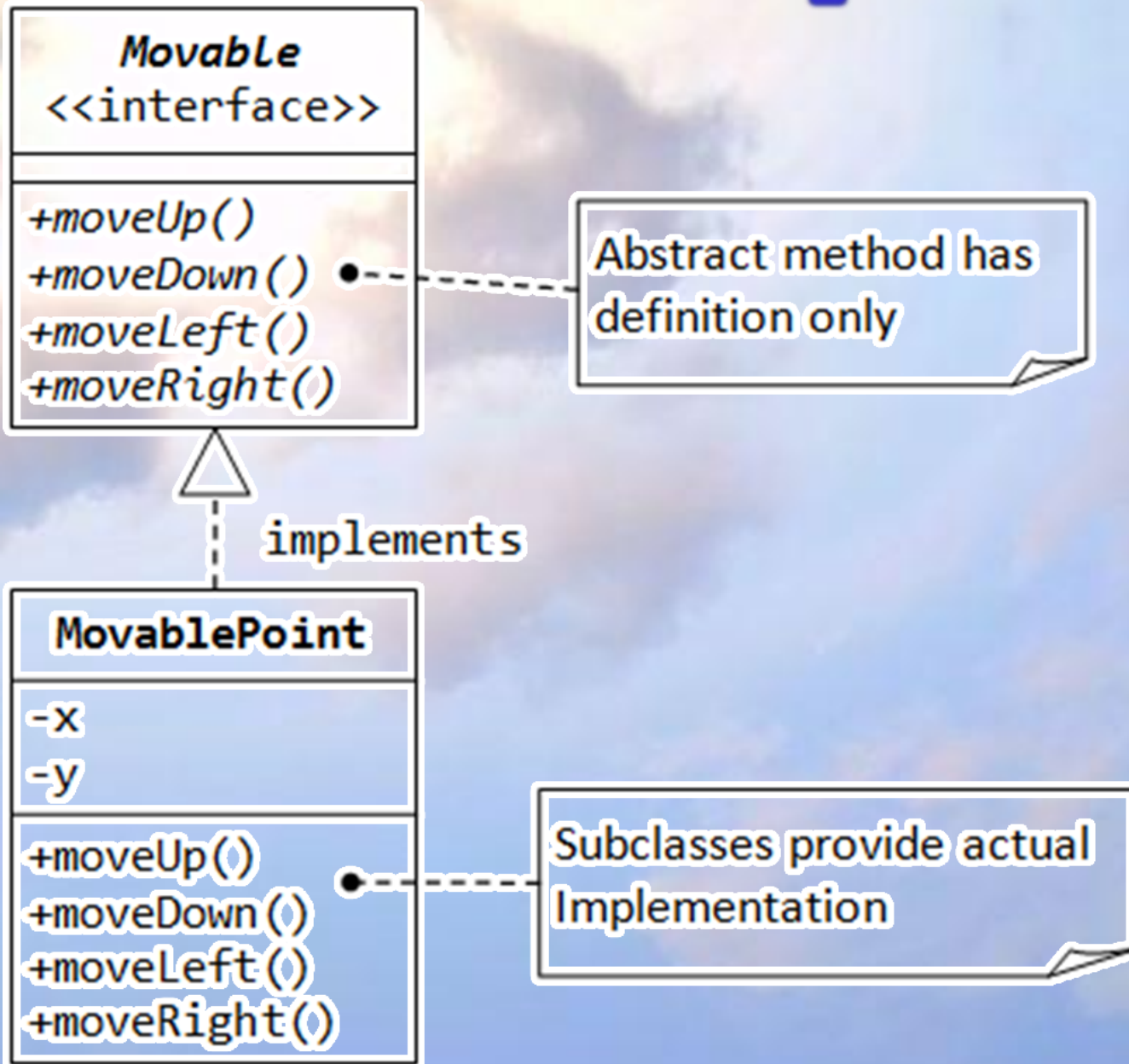
- An interface is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types.
- Method bodies exist only for default methods and static methods.
- Sometimes called as *pure abstract class* (but this is misleading because the keyword **abstract** is not used in interface!)
- Interfaces cannot be instantiated — they can only be implemented by classes or extended by other interfaces.
- Defining an interface is similar to a abstract class but
 - **No class keyword**
 - Method signatures (declarations) have no braces and terminated with semicolon
- Interfaces are typically used as *contracts* to control how classes are created and defined.

Interface example

```
public interface OperateCar {  
    // constant declarations, if any  
    // method signatures  
    // An enum with values RIGHT, LEFT  
    int signalTurn(Direction direction, boolean signalOn);  
    int getRadarFront(double distanceToCar, double speedOfCar);  
    .....  
    // more method signatures  
}
```

```
public class OperateBMW760i implements OperateCar {  
    // the OperateCar method signatures, with implementation -  
    // for example:  
    int signalTurn(Direction direction, boolean signalOn) {  
        // code to turn BMW's LEFT turn indicator lights on  
        // code to turn BMW's LEFT turn indicator lights off  
        // code to turn BMW's RIGHT turn indicator lights on  
        // code to turn BMW's RIGHT turn indicator lights off  
    }  
    // other members, as needed -- for example, helper classes  
    // not visible to clients of the interface  
}
```

Interface example 2



```
public interface Movable {  
    // methods to be implemented by subclasses  
    public void moveUp();  
    public void moveDown();  
    public void moveLeft();  
    public void moveRight();  
}
```

```
public class MovablePoint implements Movable {
    // Instance variables - (x, y) coordinates of the point
    private int x;
    private int y;
    public MovablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public String toString() {
        return "Point at (" + x + ", " + y + ")";
    }

    @Override
    public void moveUp() { y--; }

    @Override
    public void moveDown() { y++; }

    @Override
    public void moveLeft() { x--; }

    @Override
    public void moveRight() { x++; }
}
```

Abstract or interface?

- Both work to provide similar functionality - You cannot instantiate them, and they may contain a mix of methods declared with or without an implementation.
- With abstract classes, you can declare variables that are not static and final, and define public, protected, and private concrete methods.
- With interfaces, all variables are public, static, and final, and all methods that you declare or define (as default methods) are public. If you don't say "public" in an interface, it'll still be public.
- You can extend from an abstract but you implement from an interface
- You can only extend from one (1) abstract class but you can implement from multiple interfaces.

- Consider using abstract classes if any of these statements apply to your situation:
 - You want to share code among several closely related classes.
 - You expect that classes that extend your abstract class have many common methods or fields, or require access modifiers other than public (such as protected and private).
 - You want to declare non-static or non-final fields. This enables you to define methods that can access and modify the state of the object to which they belong.
- Consider using interfaces if any of these statements apply to your situation:
 - You expect that unrelated classes would implement your interface. For example, the interfaces Comparable and Cloneable are implemented by many unrelated classes.
 - You want to specify the behavior of a particular data type, but not concerned about who implements its behavior.
 - You want to take advantage of multiple inheritance of type.

final

- **final** is a keyword which works like **const** but for inheritance purposes
- A **class** declared as **final** can NOT be subclassed any further. Many existing classes provided by Java are final (e.g. **System**, **String** etc)
- A method declared as **final** can NOT be overridden or hidden in a subclass (but may be overloaded)
- A variable declared as **final** may only be initialized once (during declaration) and cannot be changed during run-time

Revision!

- What you need to do on your own
 - Revise how class inheritance works in C++ if you have forgotten and then refer the concepts in Java
 - Implement your own **abstract** and **interface** classes and inherit or extend new object classes from them – experiment to identify the differences between them